

Vtable hooking 실습 및 추가 과제

오늘 할 실습은 이전과 비슷하게 일종의 dll injection인데 그 중에서도 virtual table hooking 실습을 하려고 한다.

우선 기본적인 목표는 3d게임에서 벽과 주사위 사이의 깊이 정보를 옆에서 벽의 반대쪽에서도 주사위를 투과해서 볼 수 있게 하는 것이다.

이를 위해서 필요한 이론을 간략하게 정리하면 우선 api hooking에는 vtable hooking(high level), assembly hooking(low level)이 존재하며 실제로 후킹을 할 때에는 우선 inject하려는 dll의 이름을 다른 프로세스에 작성 후에 새 스레드를 만들고 LoadLibrary같은 함수로 그 dll을 실제로 inject하는 게 기본적인 hooking을 하는 방법이다.

또한 vtable에는 각 함수의 이름에 따른 함수의 주소가 적힌 배열로 virtual void 함수를 호출하는 경우에 대해서 이 테이블을 통해 함수를 호출하기 때문에 이 테이블의 주소가 적힌 값을 가져오면 된다.

또한 이번에 다루는 게임의 코드에서는 virtual void 함수도 나오게 되는데 이 함수는 호출하는 객체의 주소를 첫 번째 인자로 받는다는 특징이 존재한다.

이를 위해서는 일단 hooking의 목표부터 정해야 하는데 main을 분석해 보면 drawIndex의 함수가 실제로 그림을 그리는 함수라는 것을 알 수가 있고 따라서 이 함수를 hooking하면 된다.

이를 위해서 먼저 installVtableHook() 함수를 만들고 나서 가장 먼저 해야하는 것은 vtable의 주소를 구하고 또한 거기서 drawindexed 함수를 ida를 통해서 추적을 해서 찾은 후에 이를 우리가 원하는 함수로 바꿔치기를 해야한다. 우선 draw_indexed 함수를 추적하면

```
g_ctx->IASetIndexBuffer(g_wallIB, DXGI_FORMAT_
SetCB(XMMatrixScaling(1.5f, 1.5f, 1.0f), viewF
g_ctx->DrawIndexed(6, 0, 0);

// --- 2) 저 (변 드 3-2) : DrawIndexed(36) ---
```

이와 같이 사용이 되고 있음을 파악할 수 있고 g_ctx를 추적하면

```
static ID3D11Device*      g_dev      = nullptr;
static ID3D11DeviceContext* g_ctx     = nullptr;
```

와 같이 ID3D11DeviceContext라는 이름의 vtable에서 정보를 가지고 온다는 사실을 파악할 수 있다 이를 ida에서 찾아보면 view -> open subviews -> type library 을 통해 구조체와 정보를 보는 대에서 이를 찾아보면 이렇게 vtable의 12번째 위치에 존재한다는 걸 알 수 있다.

```
00000000 struct ID3D11DeviceContext1Vtbl // sizeof=0x430
00000000 {
00000000     HRESULT (__stdcall *QueryInterface)(ID3D11DeviceContext1 *This,
00000008     ULONG (__stdcall *AddRef)(ID3D11DeviceContext1 *This);
00000008     ULONG (__stdcall *Release)(ID3D11DeviceContext1 *This);
00000010     void (__stdcall *GetDevice)(ID3D11DeviceContext1 *This,
00000018     HRESULT (__stdcall *GetPrivateData)(ID3D11DeviceContext1 *This,
00000020     HRESULT (__stdcall *SetPrivateData)(ID3D11DeviceContext1 *This,
00000028     HRESULT (__stdcall *SetPrivateDataInterface)(ID3D11DeviceContext1 *This,
00000030     void (__stdcall *VSSetConstantBuffers)(ID3D11DeviceContext1 *This,
00000038     void (__stdcall *PSSetShaderResources)(ID3D11DeviceContext1 *This,
00000040     void (__stdcall *PSSetShader)(ID3D11DeviceContext1 *This,
00000048     void (__stdcall *PSSetSamplers)(ID3D11DeviceContext1 *This,
00000050     void (__stdcall *VSSetShader)(ID3D11DeviceContext1 *This,
00000058     void (__stdcall *VSSetShader)(ID3D11DeviceContext1 *This,
00000060     void (__stdcall *DrawIndexed)(ID3D11DeviceContext1 *This,
```

```

.data:000000014002FC74 align 0
.data:000000014002FC78 ; ID3D11DeviceContext *ppImmediateContext
.data:000000014002FC78 ppImmediateContext dq 0 ; DATA
.data:000000014002FC78 ; sub_
.data:000000014002FC80 ; ID3D11Device *ppDevice

```

그 다음에 이제 이 vtable에서의 위치 정보를 활용해서 이 과정을 조금 더 구체적으로 말하면 우선 GetModuleHandle(null)을 이용해 base 주소를 구하고 그 다음에 drawindex함수가 어느 vtable에 속해 있는지 ida를 통해서 vtable의 offset을 알아낸 후 그 둘을 더하면 vtable의 주소를 구할 수 있다. 또한 이 때 vtable을 ppImmediateContext를 사용해서 접근했기 때문에 이것의 상대주소를 더해주면 된다.

그 다음에 그 함수의 주소를 알았으니 이걸 바꿔치기를 바로하려고 하면 안되는데 그 이유는 vtable이 기본적으로 read mode이기 때문이고 따라서 이를 write모드로 바꾼 후에 바꿔치기를 한다.

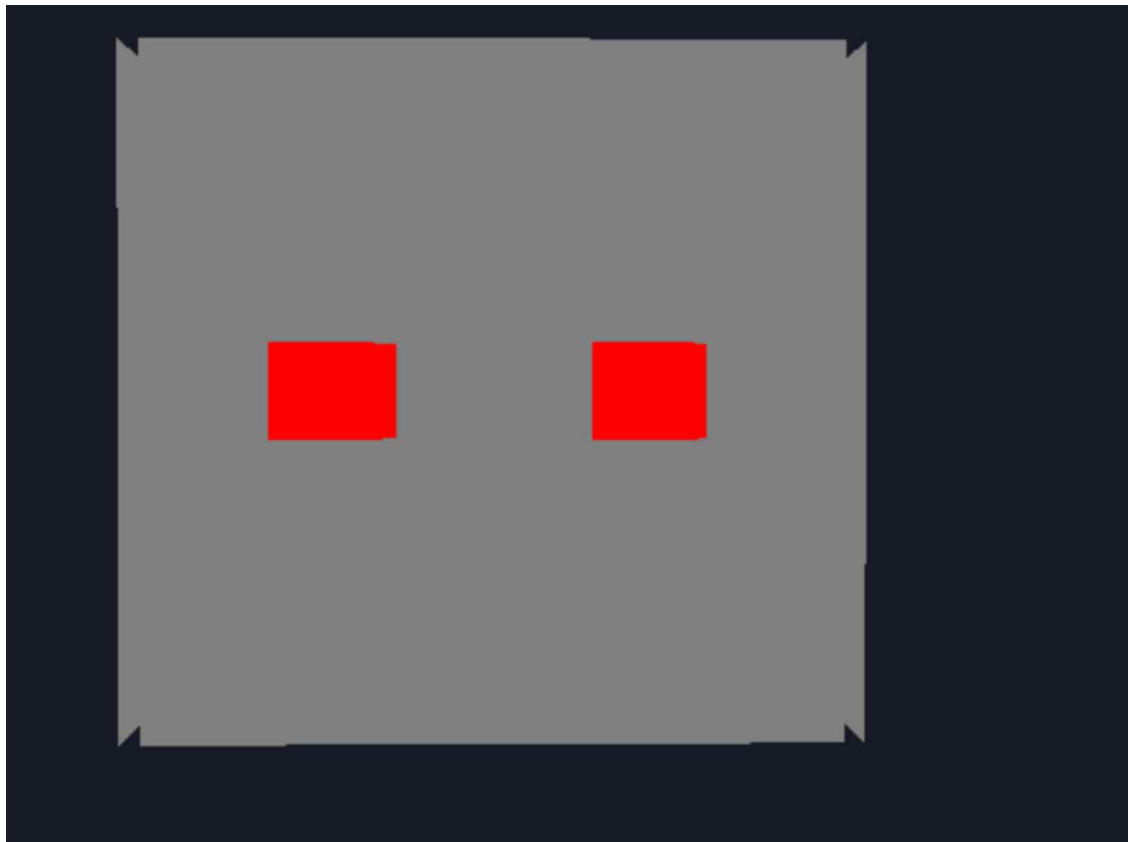
이 과정을 조금 더 디테일하게 payload.c의 installVTableHook함수에 주석으로 내용을 담아놔으니 이를 확인하면 된다.

이제 바꿔치기를 했으니 실제 inject할 함수를 payload.c에 작성을 해야 한다, 우선 지금 나의 목표는 주사위의 깊이 정보를 없애는 것이 목표이기 때문에 주사위일 때만 깊이 정보를 수정하기 위해서 주사위를 보니 index count가 36개인 고유한 특징을 파악하고 36개가 아닌 경우에는 기존의 drawindex함수를 그대로 호출을 하고 맞는 경우에만 깊이 정보를 바꾸면 된다.

이를 위해서 바꿔치기 이전에 전역 포인터로 백업을 하고 깊이 정보를 `ctx->setdepth`와 같은 함수로 제거한 후에 다시 backup본을 가져오고 원래 함수를 호출하고 old state로 복귀를 한다.

이렇게 작성을 하면 이제 주사위가 벽 뒤에 있더라도 깊이 정보가 없어졌기 때문에 주사위가 보이는 걸 확인할 수 있다. 이를 좀 더 응용한다면 매우 사기적인 투과핵을 만들 수 있게 되는 것이다.

그 다음 이제 기본적인 분석은 끝났고 그 다음 마지막으로 이게 진짜 보이는 지 더 명확히 확인하기 위해서 양쪽 사이드에도 벽을 만들어 보면 아래와 같이 벽을 투과하고 있다는 걸 더 확실하게 확인할 수 있다.



벽을 만든 방식은 기존의 있는 벽의 형태는 그대로 사용하고

```
SetCB(XMMatrixScaling(1.5f, 1.5f, 1.0f), viewProj, XMFLOAT4(0.5f, 0.5f, 0.5f, 1)); // 회색
```

```
g_ctx->DrawIndexed(6, 0, 0);
```

이 부분만 left_wall, right_wall을 선언하고 벽을 기존의 front wall을 90도를 돌리고(이건 이미 main에 존재한다.) 그 다음에 왼쪽 벽은 x좌표쪽으로 오른쪽 벽은 x좌표에 양수를 넣고 z값은 앞의 벽이 $z = 0$ 에 있고 z의 길이가 3임으로 $z = 1.5$ 로 설정해 0~3.0범위 내에 존재하게 만들었다.

//좌우 벽 선언

```
XMMATRIX left_wall = XMMatrixScaling(1.5f, 1.5f, 1.0f) * XMMatrixRotationY(XM_PIDIV2) * XMMatrixTranslation(-1.59f, 0.0f, 1.5f);
```

```
XMMATRIX right_wall = XMMatrixScaling(1.5f, 1.5f, 1.0f) * XMMatrixRotationY(XM_PIDIV2) * XMMatrixTranslation(1.59f, 0.0f, 1.5f);
```

//좌측의 벽

```
SetCB( left_wall, viewProj, XMFLOAT4(0.5f, 0.5f, 0.5f, 1)); // 회색
```

```
g_ctx->DrawIndexed(6, 0, 0);
```

//우측의 벽

```
SetCB(right_wall, viewProj, XMFLOAT4(0.5f, 0.5f, 0.5f, 1)); // 회색
```

```
g_ctx->DrawIndexed(6, 0, 0);
```

